Article

Evaluating the Impact of Adapter-Based Fine-Tuning on Structured Parsing Performance in Large Language Models

Ratomir Karlović [1], Luka Sever [1], Sandi Berossi Šegota [1] and Ivan Lorencin [1]

[1] Faculty of Informatics, Juraj Dobrila University of Pula, Zagrebačka 30, 52100 Pula, Croatia; ratomir.karlovic@unipu.hr (R.K.), luka.sever@unipu.hr (L.S.), sandi.baressi.segota@unipu.hr (S.B.Š.), ivan.lorencin@unipu.hr (I.L.)
* Correspondence: ivan.lorencin@unipu.hr;

Abstract

Recent advances in large language models (LLMs) highlight two dominant strategies for performance improvement: prompt engineering and fine-tuning. While prompt design can significantly influence model output, it remains uncertain whether lightweight fine-tuning methods,such as adapter-based training, offer meaningful advantages for structured, domain-specific tasks. This study builds on prior research comparing three prompting strategies for natural-language command parsing into JSON schemas. Expanding that framework, the current work investigates how adapter-based fine-tuning, where most model parameters are frozen and only small adapter modules are trained, affects model accuracy, consistency, and robustness. The experiment uses the same controlled shopping-cart parsing task and dataset of 1000 synthetic commands to ensure direct comparability. Results will quantify the trade-off between computational cost and performance gains, offering evidence-based insights into whether fine-tuning is a justified investment compared to advanced prompt engineering. The expected contribution is a clear, empirical framework for deciding when fine-tuning meaningfully enhances LLM utility in applied natural-language understanding.

Keywords: LLM, Adapters Fine-Tuning, Prompt Engineering, Natural Language Understanding, JSON Parsing, Structured Output, Model Performance

1. Introduction

The rapid evolution of Large Language Models (LLMs) has fundamentally transformed the landscape of Natural Language Processing (NLP), enabling advanced capabilities in reasoning, content generation, and semantic understanding [1]. While early applications focused primarily on open-ended text generation, modern production environments increasingly require LLMs to function as reliable semantic parsers capable of mapping unstructured user intent into rigorous, machine-readable formats such as JSON or SQL [2]. This capability is particularly critical in domain-specific applications, such as e-commerce controllers, where valid schema adherence is a prerequisite for system functionality. A primary challenge in deploying these systems is the "optimization dilemma": choosing between inference-time strategies and model training. As highlighted in our previous survey [3], while prompt engineering offers a lightweight "black-box" approach to adaptation, it often struggles with robustness when constrained to rigid schema definitions. Conversely, full-parameter fine-tuning yields high precision but incurs prohibitive computational costs and memory requirements [4]. To address this, Parameter-Efficient

Received:
Revised:
Accepted:
Published:

 https://doi.org/10.3390/1010000

Fine-Tuning (PEFT) techniques, such as Low-Rank Adaptation (LoRA), have emerged as a middle ground, freezing pre-trained weights and injecting trainable rank-decomposition matrices to approximate full fine-tuning performance with a fraction of the resources [5]. To address these challenges, this study evaluates the cost-benefit ratio of fine-tuning against prompting for high-fidelity semantic parsing. Building upon the theoretical frameworks established in our prior work [3], we implement a controlled experimental design using the LLaMA-3-8B model. The system compares two distinct optimization paradigms: Advanced Prompt Engineering and Adapter-Based Fine-Tuning, utilizing LoRA and its quantized variant (QLoRA) to update the model's internal representations [6]. This setup allows for a direct comparison of how each method handles the rigorous constraints of mapping natural language commands to structured JSON outputs. The main aim of this research is to identify the "break-even point" where the computational investment of training adapters yields a statistically significant advantage over advanced prompting. The main contribution of this study is a structured empirical framework for benchmarking LLaMA-3 on structured parsing tasks, providing evidence-based insights into whether the resource consumption of fine-tuning is justified by gains in accuracy and robustness. The evaluation reveals that while prompting is sufficient for simple intents, adapter-based methods significantly outperform in handling complex, multi-intent schemas, particularly when targeting all linear layers with optimized hyperparameters.

2. Background and Related Works

The rapid growth of large language models (LLMs) has necessitated strategies for adapting them efficiently to downstream tasks. Zhiqiang Hu et al. [5] introduced LLM-Adapters, a framework for parameter-efficient fine-tuning (PEFT) that integrates Series adapters, Parallel adapters, Prefix-Tuning, and LoRA. Their study demonstrated that smaller LLaMA models (7B and 13B) equipped with optimized adapters could match or surpass the performance of much larger models such as ChatGPT and GPT-3.5, particularly on structured reasoning tasks using datasets like Math10K and Commonsense170K. These findings suggest that adapter-based PEFT can improve structured task accuracy compared to advanced prompting strategies.

Ding et al. [7] proposed a unified framework called delta-tuning, which adjusts only a small fraction of model parameters. Through extensive evaluation on over 100 NLP tasks, they found that delta-tuning achieved performance comparable to full fine-tuning while significantly reducing GPU memory usage and accelerating backpropagation. Larger models not only benefited more from minimal parameter updates but also converged faster, reinforcing the potential of adapter-based methods to enhance task-specific performance efficiently.

Esmaeili et al. [8] conducted empirical studies on PEFT for code-specific LLMs, examining LoRA, Compacter, and IA3. Their findings showed that LoRA consistently outperformed other adapters in code summarization and generation, occasionally exceeding full fine-tuning on low-resource languages like R. The study highlighted that the number of trainable parameters influenced functional correctness more than the adapter architecture itself, further supporting the effectiveness of PEFT in improving structured outputs in specialized domains.

Razuvayevskaya et al. [9] compared parameter-efficient techniques and full fine-tuning for multilingual news classification. Using XLM-RoBERTa Large, they showed that LoRA and bottleneck adapters could reduce trainable parameters by 140–280 times while maintaining competitive accuracy. Their results suggested that PEFT strategies can outperform traditional prompting approaches, especially when sufficient source data is available.

Shin et al. [10] assessed the trade-offs between prompt engineering and fine-tuning for automated code tasks. Their evaluation of GPT-4 against seventeen fine-tuned baselines revealed that while task-specific prompting sometimes outperformed automated methods in code summarization, fine-tuned models were generally superior in code generation. Human-in-the-loop conversational prompting further enhanced outcomes, demonstrating that adapter-based fine-tuning can stabilize and improve output quality over prompting alone.

Ming Gong et al. [11] developed structure-learnable adapters, which introduced differentiable gating and sparsity control to dynamically optimize adapter placement and activation paths. Experiments on the Multi-Task NLU Benchmark indicated that this approach not only achieved high accuracy with minimal parameters but also reduced output variability and maintained robustness under noisy conditions, supporting the notion that adapter fine-tuning can yield more consistent outputs than prompting.

Fang and Ye [12] proposed RFedLR, a robust PEFT framework for federated learning. By selectively updating noise-sensitive parameters and dynamically weighting client updates, RFedLR achieved up to 10.15% accuracy improvements over state-of-the-art baselines under high-noise conditions while using only 0.1754% of the trainable parameters. These results reinforce that adapter-based fine-tuning provides more stable and reliable outputs than prompt-based methods, even in decentralized and noisy environments.

Shuo Chen et al. [13] benchmarked eleven adaptation methods on vision-language models under textual and visual corruptions. They found that parameter-efficient adapters exhibited higher resilience to input perturbations than full fine-tuning or prompting, although no single method dominated across all datasets. Increasing adaptation data or parameter size did not guarantee robustness, highlighting the importance of careful PEFT design to enhance model stability.

Chen et al. [14] evaluated prompt engineering for industrial document processing tasks and found that few-shot learning improved reasoning capabilities over zero-shot prompts. However, adapter-based PEFT consistently offered more reliable and less variable outputs, particularly in complex scenarios with noisy OCR inputs.

Kaijie Zhu et al. [15] introduced PromptRobust, a benchmark for adversarial prompt evaluation across multiple LLMs. Their results showed that word-level adversarial prompts caused a 39% average performance drop, and few-shot prompts exhibited better robustness than zero-shot prompting. The study illustrates that adapter fine-tuning can improve resilience to prompt-based perturbations.

Jaehyung Kim et al. [16] presented ROAST, combining adversarial perturbations with selective training to improve robustness across sentiment classification and entailment tasks. ROAST yielded average improvements of 18.39% and 7.63% over state-of-the-art baselines, demonstrating that adapter fine-tuning can mitigate input noise effects more effectively than prompting.

Pei et al. [17] proposed SelfPrompt, which autonomously evaluates LLM robustness using domain-constrained knowledge graphs and adversarial prompt generation. Their findings indicated that domain-specific adapter fine-tuning enhances robustness in specialized fields like medicine and biology, outperforming prompting strategies alone.

Lin Mu et al. [18] introduced Robustness of Prompting (RoP), a parameter-free method for improving LLM resilience through error correction and guidance prompts. While effective, experiments confirmed that adapter fine-tuning provides superior stability and transferability across architectures, emphasizing the practical value of PEFT in noisy real-world environments.

Sajjadi Mohammadabadi et al. [19] surveyed LLM architectures and adaptation methods, emphasizing the efficiency of PEFT, instruction tuning, and RLHF. They noted that adapter-based fine-tuning can deliver significant performance gains relative to prompting, justifying its computational cost in applied settings.

Trad and Chehab [20] studied phishing detection LLMs and found that while refined prompting improved performance, fine-tuned models achieved higher F1-scores and superior reliability, demonstrating that the cost of PEFT is justified when high precision is required.

Pornprasit and Tantithamthavorn [21] evaluated LLM-based code review, showing that fine-tuning GPT-3.5 achieved 73–74% higher Exact Match rates than prompting approaches. Their study confirmed that adapter-based fine-tuning can deliver meaningful gains even with modest training data.

Wang et al. [22] introduced COSMOS, a framework for predicting adaptation outcomes with minimal trials. Their results indicated that fine-tuning consistently outperformed in-context learning in medium to high-cost scenarios, supporting the view that adapter PEFT justifies its resource investment by providing predictable and superior performance.

Based on the cumulative evidence from these studies, the overarching goal of this work is to investigate how adapter-based parameter-efficient fine-tuning (PEFT) compares to advanced prompting in terms of structured task accuracy, output stability, robustness, and cost-effectiveness. Accordingly, the following research hypotheses are formulated:

- H1: Parameter-efficient fine-tuning methods improve structured output generation performance compared to prompt-based adaptation strategies.
- H2: LoRA-based adapters achieve higher structured extraction accuracy than other parameter-efficient tuning techniques such as IA3.
- H3: Larger models benefit less from advanced prompt engineering than smaller models, indicating that parameter-efficient adaptation can compensate for differences in model scale.
- H4: Prompt engineering alone can achieve high structured output accuracy, but parameter-efficient fine-tuning provides more consistent and near-perfect results.

3. Methodology

This section describes the experimental framework designed to evaluate the trade-offs between parameter-efficient fine-tuning (PEFT) and advanced in-context learning (ICL) for structured information extraction. The primary objective is to measure performance, structural reliability, and generalization capability of large language models (LLMs) when converting unstructured natural language instructions into strictly formatted JSON objects. The methodology extends the comparative benchmarking approach of [3] to a controlled structured parsing domain.

3.1. Task Definition

The experimental task focuses on Natural Language to Structured Command conversion, a core component of automated commerce and inventory systems. Given a free-form shopping instruction, the model must output a valid JSON object with the following schema:

```
{
  "action": "...",
  "quantity": "...",
  "product": "..."
}
```

Each instance requires extraction or inference of three attributes:

The experimental task focuses on the "Natural Language to Structured Command" conversion, a critical component in automated inventory and e-commerce systems. To rigorously evaluate the effectiveness of both prompting and adapter-based fine-tuning, we constructed a controlled dataset comprising 10,000 annotated natural-language shopping instructions. To ensure high-quality and diverse data, samples were synthetically generated using a teacher-model paradigm.

Each instance requires the model to extract or infer three specific attributes and map them deterministically to a fixed JSON schema:

1. Action: Binary classification over {add, remove}.
2. Product: Named entity extraction identifying the canonical product name.
3. Quantity: Integer extraction with a default value of 1 when no explicit quantity is provided.

The mapping from natural language to JSON is deterministic, enabling objective evaluation of both semantic correctness and structural integrity.

3.2. Dataset Construction

The dataset consists of 12,000 synthetically generated but linguistically diverse shopping instructions, created in collaboration with industry partners to reflect realistic user interaction patterns.

Although synthetically produced, the corpus incorporates substantial variation to prevent surface-form memorization. Diversity mechanisms include:

3.2.1. Dataset Splitting for Generalization Assessment

To balance computational feasibility with sufficient diversity for adapter specialization, the 10,000 examples were partitioned into three disjoint subsets:

- Verb paraphrasing (e.g., add, insert, remove, delete, take out)
- Explicit and implicit quantity expressions
- Variable grammatical constructions
- Multi-word paraphrases and lexical substitutions

Each example is paired with a normalized ground-truth JSON annotation containing the three labeled attributes.

3.3. Data Splits

To ensure reliable model evaluation and prevent data leakage during training, the dataset was divided into three disjoint subsets for training, validation, and testing. The split was designed to provide sufficient data for adapter optimization while preserving a representative evaluation set for assessing model generalization.

The corpus was partitioned as follows:

- Training set: 8,000 examples used for adapter parameter optimization during supervised fine-tuning.
- Validation set: 2,000 examples used for monitoring model convergence, tuning training dynamics, and preventing overfitting.
- Test set: 2,000 disjoint examples reserved exclusively for the final evaluation of model performance.

The validation and test sets remain completely unseen during adapter training. The test set additionally contains novel paraphrasing patterns and lexical combinations to evaluate the model's ability to generalize beyond the training distribution.

3.4. Model Selection

To analyze the effect of model scale and architectural variation, we evaluate a heterogeneous suite of LLMs across parameter sizes and training paradigms. We have selected three models that represent a range of scales and architectural designs, all of which are compatible with adapter-based fine-tuning:

Table 1. Evaluated models and their parameter counts

Name	Size
Llama 3.1	8B
Qwen 3	4B
Llama 3.2	3B

All models are evaluated under identical inference conditions with temperature set to 0 to eliminate stochastic variance.

3.5. Parameter-Efficient Fine-Tuning

For each base model, we compare three configurations:

1. Base Model (Prompting Only): No parameter updates; task performed via structured prompt engineering.
2. LoRA Adapter: Low-Rank Adaptation modules trained on the 8,000-example training set.
3. IA^3 Adapter: Infused Adapter by Inhibiting and Amplifying Inner Activations trained under identical conditions.

Adapters are implemented using the PEFT framework within the Hugging Face Transformers ecosystem. Hyperparameters, tokenizer configuration, and preprocessing steps are held constant across experiments to ensure fair comparison. No hyperparameter search is performed.

Training is conducted using multi-GPU Distributed Data Parallel (DDP).

QLoRA is not employed in this study.

3.6. Evaluation Protocol

All configurations are evaluated on the same held-out test set of 2,000 examples.

A prediction is considered correct only if:

- All three fields (action, quantity, product) exactly match the ground truth.
- The output is valid JSON and can be successfully parsed.

If the model fails to produce valid JSON, the prediction is counted as incorrect.

Performance is measured using Precision, Recall, and F1 Score under strict exact-match criteria. F1 Score serves as the primary comparison metric.

3.7. Comparative Analysis

For each model, we report results in the following format:

Model	Evaluated approaches	Approach
...	...	...

This structure enables direct comparison between prompt-based inference and adapter-based specialization across model scales.

3.8. Adapter-Based Fine-Tuning (LoRA)

For the fine-tuning paradigm, we employed Low-Rank Adaptation (LoRA). This technique was chosen for its ability to maintain the underlying general knowledge of the base model while specializing its output structure for JSON parsing with minimal computational overhead.

The LoRA approach modifies the pre-trained weight matrices $W_0 \in \mathbb{R}^{d \times k}$ by adding a low-rank decomposition BA, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$. During the forward pass, the output of the adapter is scaled by a factor of $\frac{\alpha}{r}$ to balance the retention of pre-trained information with the targeted domain adaptation. The forward pass for a given input x is thus represented as:

$$h = W_0 x + \left(\frac{\alpha}{r}\right) \Delta W x = W_0 x + \left(\frac{\alpha}{r}\right) BAx \quad (1)$$

In this study, we targeted all linear modules within the transformer architecture to ensure that the adapter captured the structural and semantic constraints required for reliable JSON generation. The adaptation was applied to both the attention mechanism and the feed-forward network layers, specifically:

- `q_proj` – Query projection layer that generates query vectors used to compute attention scores between tokens.
- `k_proj` – Key projection layer that produces key vectors used to determine the relevance of tokens during attention computation.
- `v_proj` – Value projection layer that encodes the information aggregated through the attention mechanism.
- `o_proj` – Output projection layer that combines the results of multiple attention heads and maps them back into the model's hidden representation space.
- `gate_proj` – Gating projection within the feed-forward network that regulates the activation flow in the nonlinear transformation stage.
- `up_proj` – Expansion projection layer that increases the dimensionality of hidden representations within the feed-forward block.
- `down_proj` – Compression projection layer that reduces the expanded representation back to the original hidden size.

The specific LoRA configuration utilized during fine-tuning included:

- Rank (r): 32. A higher rank was selected to provide sufficient parameter capacity for the complex semantic mapping required in structured parsing tasks.
- Alpha (α): 64. The scaling factor was set to $\alpha = 2r$, a heuristic commonly used to stabilize training and improve convergence when employing higher ranks.
- Dropout: 0.05, applied to the adapter layers as a regularization measure to mitigate overfitting.
- Precision and Compute: Training was performed without 4-bit quantization, utilizing BFloat16 (`bf16`) mixed precision and TensorFloat-32 (`tf32`) optimizations supported by NVIDIA A100 GPUs to accelerate matrix multiplications.

3.9. Infused Adapter by Inhibiting and Amplifying Inner Activations (IA3)

While most fine-tuning methods focus on adding new structural layers or modifying a model's global weights, IA3 adopts a minimalist approach by selectively inhibiting or amplifying existing internal signals. Instead of introducing heavy new components, IA3 integrates three specific learnable vectors that act as signal modulators within the transformer's architecture[23].

3.9.1. The Signal Modulation Mechanism

The IA3 method utilizes a mathematical operation known as a Hadamard product to precisely scale specific pieces of information as they flow through the system. The function of these adjustable modulators is divided into two primary roles:

- Attention Modulators (l_k and l_v): These vectors enable the model to prioritize specific parts of an input sentence[cite:]. For example, in a shopping-cart command, these modulators can amplify the signal of a product name like "apples" while suppressing irrelevant filler words that do not contribute to the final structured output.
- Feed-Forward Modulator (l_{ff}): This vector modifies the signals within the model's memory center, specifically the feed-forward network. It assists the model in correctly categorizing the user's intent, such as distinguishing between an instruction to "add" an item or "remove" it from the digital cart.

3.9.2. Key Advantages for Practical Deployment

The IA3 approach offers several distinct benefits for high-precision tasks under resource-constrained conditions:

- Extreme Parameter Efficiency: IA3 represents a highly lightweight version of fine-tuning. It typically updates significantly fewer settings than other popular methods like LoRA, often modifying less than 0.01% of the total model parameters.
- Zero Performance Latency: Because these signal modulations can be mathematically merged into the original model weights after training, the customized model operates at the same speed as the original version. No additional computational overhead is introduced during the inference phase.
- Competitive Accuracy: Despite its small footprint, IA3 has been demonstrated to match or even outperform significantly more complex fine-tuning methods on specialized reasoning and mathematical tasks.

By utilizing these precise signal modulators, IA3 enables the model to achieve professional-grade precision in structured parsing without the heavy computational costs typically associated with larger fine-tuning strategies [23].

3.9.3. Optimization and Training Dynamics

The Supervised Fine-Tuning (SFT) phase was executed over 3 epochs with a maximum sequence length of 1024 tokens. To optimize memory footprint during backpropagation, gradient checkpointing was enabled. The training pipeline utilized a fused AdamW optimizer (`adamw_torch_fused`) with a peak learning rate of 2×10^{-4}, a weight decay of 0.01, and a warm-up ratio of 3% to ensure early-stage stability. An effective batch size of 32 was maintained by utilizing a per-device batch size of 8 combined with 4 gradient accumulation steps.

3.10. Prompt Engineering Strategies

To establish a baseline for In-Context Learning (ICL), we implemented three escalating levels of prompt complexity:

1. Minimal Instruction (Zero-Shot): A concise prompt defining the role of the model and the required JSON schema.
2. Extended Prompt: This strategy includes detailed edge-case instructions, emphasizing the default quantity logic and strict "no-prose" constraints to prevent conversational "chatter."
3. Few-Shot Prompting: Building upon the Extended Prompt, this version includes [Insert Number] high-quality examples of natural language inputs and their cor-

responding JSON outputs, providing a semantic and structural blueprint for the model.

3.11. Evaluation Metrics

The models were evaluated on two primary dimensions to assess both syntactic integrity and semantic accuracy.

3.11.1. JSON Validity Rate

Before measuring accuracy, we assess if the output is a "well-formed" JSON object. Any output that includes conversational filler, markdown errors, or fails standard library parsing (e.g., Python's `json.loads()`) is categorized as a failure. This metric directly evaluates the model's susceptibility to structural hallucination.

3.11.2. Structured F1 Score

For all valid JSON outputs, we calculate the F1 score. This requires an exact match between the predicted value and the ground truth for each of the three fields (Action, Product, Quantity). The field-level F1 score is defined as the harmonic mean of precision (P) and recall (R):

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} \tag{2}$$

We report the macro-averaged F1 across all fields to provide a singular, robust measure of parsing performance.

4. Results and discussion

This section evaluates the performance of three large language models:Llama 3.1 8B, Llama 3.2 3B, and Qwen3 4B under multiple adaptation strategies, including minimal instruction prompting, extended prompting, few-shot prompting, and parameter-efficient fine-tuning methods (IA^3 and LoRA). The evaluation was conducted on a dataset of 2000 samples, with performance measured using exact-match precision, recall, F1-score, and accuracy for structured JSON output generation. Table 2 summarizes the overall results across all evaluated approaches.

Table 2. Overall results across all evaluated approaches

Model	Exact match accuracy	F1 score	Macro-Averaged F1	JSON Validity Rate	Approach
Llama 3.1 8B	0.965	0.982	0.985	1.000	Minimal Instruction Prompting
Llama 3.2 3B	0.927	0.962	0.983	0.998	Minimal Instruction Prompting
Qwen3 4B	0.992	0.996	0.998	1.000	Minimal Instruction Prompting
Llama 3.1 8B	0.971	0.985	0.990	1.000	Extended Prompting
Llama 3.2 3B	0.940	0.969	0.984	0.999	Extended Prompting
Qwen3 4B	0.989	0.994	0.997	1.000	Extended Prompting
Llama 3.1 8B	0.944	0.971	0.988	1.000	Few-Shot Prompting
Llama 3.2 3B	0.942	0.970	0.986	1.000	Few-Shot Prompting
Qwen3 4B	0.949	0.974	0.989	0.997	Few-Shot Prompting
Llama 3.1 8B	1.000	1.000	1.000	1.000	LoRA
Llama 3.2 3B	0.999	0.999	1.000	1.000	LoRA
Qwen3 4B	0.999	0.999	0.999	1.000	LoRA
Llama 3.1 8B	0.960	0.980	0.992	0.997	IA^3
Llama 3.2 3B	0.976	0.988	0.996	1.000	IA^3
Qwen3 4B	0.923	0.960	0.985	1.000	IA^3

The results indicate that all models achieve high performance across prompting-based approaches, with exact match F1 scores consistently exceeding 0.96. Among prompting methods, the minimal instruction prompt produced the strongest overall performance, with the Qwen3 4B model achieving an F1 score of 0.996 and an exact match accuracy of 0.992. This result suggests that, for tasks involving structured output generation, carefully designed but concise prompts may be sufficient to guide model behavior without requiring extensive contextual examples.

Extended prompting also yielded strong results, with Qwen3 4B reaching an F1 score of 0.994, slightly below the minimal prompt configuration. In contrast, the few-shot prompting approach resulted in slightly lower performance, with F1 scores ranging between 0.970 and 0.974 across models. This decrease may indicate that introducing multiple examples increases prompt complexity and token length, which can occasionally introduce ambiguity or distract the model from the core instruction.

Across all prompt-based strategies, Qwen3 4B consistently achieved the highest performance, followed by Llama 3.1 8B and Llama 3.2 3B. This suggests that architectural differences and pretraining strategies can significantly influence the ability of models to follow structured output constraints.

https://doi.org/10.3390/1010000

4.1. Impact of Prompt Engineering Strategies

Prompt engineering strategies were evaluated to determine how different instruction styles influence structured output generation performance.

Among the tested approaches, minimal instruction prompting demonstrated the best average performance, achieving an average F1 score of 0.980 across models. The relatively high standard deviation (0.017) suggests that model size and architecture influence how effectively concise prompts are interpreted.

The extended prompt configuration, which provides additional instructions and constraints, produced an average F1 score of 0.983. While slightly higher in average performance, the improvement over minimal prompts was marginal. This indicates that once models clearly understand the task structure, additional prompt elaboration offers limited gains.

Few-shot prompting showed the lowest overall performance, with an average F1 score of 0.972 and the lowest variability (standard deviation of 0.002). This suggests that while few-shot examples provide consistent guidance across models, they do not necessarily improve peak performance for this specific task. One possible explanation is that the task—mapping natural language input into a fixed JSON schema—is already well-aligned with the pretraining objectives of modern LLMs.

An analysis of individual field performance further reveals that the product field was consistently the most challenging attribute to predict, particularly in the few-shot configuration, where its F1 score decreased to 0.949 for the best-performing model. In contrast, the action and quantity fields maintained near-perfect performance across most configurations.

These results highlight that prompt complexity does not always correlate with improved performance, especially for tasks requiring deterministic structured outputs.

4.2. Performance of Parameter-Efficient Fine-Tuning Methods

To evaluate the effectiveness of parameter-efficient adaptation methods, two widely used techniques—IA^3 and LoRA—were applied to the evaluated models.

The IA^3 adapter achieved strong results, with F1 scores ranging from 0.960 to 0.988 across models. The best IA^3 configuration was obtained using Llama 3.2 3B, which achieved an F1 score of 0.988 and exact match accuracy of 0.976. Interestingly, this result surpasses the performance of the same base model under all prompt-based configurations, indicating that lightweight parameter adaptation can significantly improve structured prediction capabilities.

However, the most notable result was achieved using LoRA adapters, which produced near-perfect performance across all evaluated models. The Llama 3.1 8B LoRA configuration achieved a perfect score, with F1, precision, recall, and accuracy all equal to 1.000, successfully predicting all 2000 samples.

Similarly, Llama 3.2 3B and Qwen3 4B LoRA models achieved F1 scores of 0.999, missing only two and three samples respectively. These results demonstrate the exceptional effectiveness of LoRA-based adaptation for structured generation tasks.

The superiority of LoRA in this setting can be attributed to its ability to introduce low-rank trainable matrices into attention layers, enabling the model to learn task-specific patterns without modifying the majority of base model parameters. This mechanism allows efficient adaptation while preserving the underlying knowledge learned during pretraining.

4.3. Comparison Between Prompt-Based and Adapter-Based Adaptation

A comparison between prompt-based strategies and parameter-efficient fine-tuning methods reveals several important insights.

First, prompt-based approaches already achieve very strong baseline performance, with F1 scores exceeding 0.96 in all tested configurations. This confirms the strong in-context learning capabilities of modern LLMs.

However, parameter-efficient fine-tuning—particularly LoRA—consistently outperformed prompt-based methods, achieving near-perfect results. This suggests that even lightweight model adaptation can significantly improve reliability in tasks requiring strictly formatted outputs.

Another important observation concerns model size efficiency. The Llama 3.2 3B model with LoRA achieved performance comparable to or better than larger models using prompting alone. This finding indicates that parameter-efficient fine-tuning may enable smaller models to achieve competitive performance while significantly reducing computational costs.

From a practical perspective, the choice between prompting and fine-tuning depends on several factors:

Table 3. Overview of prompt-based and adapter-based adaptation methods

Prompt-based methods	Adapter-based methods
No training required	Higher reliability and accuracy
Lower setup complexity	Better control over output format
Flexible for multiple tasks	Improved performance for structured prediction tasks

Thus, while prompt engineering remains a powerful and flexible technique, adapter-based tuning provides superior consistency when deterministic outputs are required.

4.4. Variability Across Models

The evaluation also reveals differences in how models respond to adaptation strategies.

Qwen3 4B consistently performed best in prompt-based scenarios, suggesting strong instruction-following capabilities and robust in-context learning performance.

In contrast, the Llama models benefited more from parameter-efficient fine-tuning, particularly with LoRA. This observation may reflect differences in pretraining data composition, tokenizer design, or alignment procedures.

Furthermore, performance differences between 3B and 8B parameter models were relatively small, especially after adapter-based tuning. This suggests that task-specific adaptation can compensate for differences in model scale, an important consideration for deployment scenarios with limited computational resources.

5. Limitations and Future Work

Although the experimental results demonstrate the effectiveness of both prompt engineering and parameter-efficient fine-tuning (PEFT) for structured output generation, several limitations of the study should be acknowledged.

First, the evaluation was conducted on a synthetic dataset consisting of 2000 samples with a relatively simple schema, containing only three output attributes (action, product, and quantity). While this controlled setup enables clear evaluation of structured extraction performance, it does not fully capture the complexity of real-world information extraction tasks. In practical applications, schemas are often significantly larger and may include hierarchical structures, optional fields, or ambiguous entities, which can introduce additional challenges for language models.

Second, the study evaluates model performance using Exact Match Accuracy, F1-score, Macro-Averaged F1, and JSON Validity Rate. These metrics effectively measure structured output correctness and formatting reliability, particularly for deterministic JSON generation tasks. However, other important evaluation aspects such as inference latency, memory consumption, and training time were not systematically analyzed. These factors may influence the practical selection of adaptation strategies in real-world deployments.

Another limitation concerns the hardware requirements associated with adapter-based fine-tuning. Although PEFT methods significantly reduce the number of trainable parameters compared to full fine-tuning, training adapters for modern LLMs still requires substantial computational resources, including GPUs with sufficient memory capacity. Hardware constraints can restrict the feasible configuration of training parameters such as batch size, learning rate schedules, adapter rank, and training duration. As a result, the final performance of adapter-based methods may vary depending on the available computational environment and the chosen hyperparameter configuration.

Related to this, the study does not perform an extensive hyperparameter optimization for adapter training. Adapter-based methods such as LoRA and IA^3 can be sensitive to parameter choices, including rank values, scaling factors, dropout rates, and learning rates. Different configurations may produce varying results even when applied to the same base model and dataset. Consequently, the performance observed in this work should be interpreted as representative of a specific configuration rather than the absolute performance ceiling of each method.

Finally, while the models achieved very high performance on the evaluated task, particularly when using LoRA adapters, the dataset characteristics and limited schema complexity may contribute to this near-perfect performance. More complex structured extraction scenarios may reveal larger performance differences between adaptation strategies.

Future work could extend this study in several directions. One promising avenue involves evaluating the proposed approaches on larger and more complex datasets, including real-world information extraction tasks with richer schemas and nested JSON structures. Such datasets would provide a more challenging benchmark for assessing the robustness and generalization capabilities of both prompting and adapter-based approaches.

Another important direction is the systematic evaluation of computational efficiency and resource requirements, including training time, GPU memory usage, and inference latency. Incorporating these factors would provide a more comprehensive comparison between prompt-based adaptation and parameter-efficient fine-tuning.

Future studies could also explore hyperparameter sensitivity and optimization for adapter-based methods, analyzing how variations in adapter rank, learning rate, and training schedules influence performance. This would help identify stable configurations that provide consistent results across different hardware environments.

Finally, additional research may investigate robustness and generalization properties, including performance under paraphrased inputs, noisy text, or multilingual settings. Understanding how adaptation strategies behave under such conditions would provide deeper insights into the practical deployment of LLM-based structured information extraction systems.

6. Conclusion

This study investigated the effectiveness of prompt engineering strategies and parameter-efficient fine-tuning (PEFT) methods for adapting large language models to structured output generation tasks. Specifically, the experiments evaluated multiple prompting configurations—minimal instruction prompting, extended prompting, and few-shot prompting—alongside two widely used adapter-based techniques, IA^3 and LoRA. The evaluation was conducted using three open-source LLMs of varying sizes, assessing their ability to transform natural language inputs into a predefined JSON schema.

The experimental results demonstrate that modern LLMs exhibit strong baseline performance in structured extraction tasks even without parameter updates. Prompt-based approaches consistently achieved high accuracy across all evaluated models, with exact match F1 scores exceeding 0.96. Among prompting strategies, minimal instruction prompts proved particularly effective, indicating that concise task descriptions are often sufficient for guiding structured generation.

Despite the strong performance of prompt engineering methods, the results show that parameter-efficient fine-tuning can further enhance reliability and accuracy. In particular, LoRA-based adapters achieved near-perfect results across all tested models, with the best configuration reaching a perfect exact match score on the evaluation dataset. IA^3 adapters also improved performance relative to several prompting configurations, although they did not match the performance achieved by LoRA. These findings support the hypothesis that lightweight parameter updates can significantly improve the consistency of structured outputs while requiring only a small fraction of trainable parameters.

Another notable observation is that parameter-efficient tuning can reduce the performance gap between models of different sizes. The smaller Llama 3.2 3B model, when adapted using LoRA, achieved performance comparable to larger models using prompt-based methods alone. This suggests that PEFT techniques can enable smaller models to achieve competitive results, which is particularly relevant for deployment scenarios with limited computational resources.

Overall, the findings highlight that while prompt engineering remains a flexible and low-cost approach for adapting LLMs, parameter-efficient fine-tuning provides superior reliability when deterministic structured outputs are required. In applications where strict output formatting and high accuracy are critical, such as information extraction, automated data processing, or API-driven systems, adapter-based methods represent a practical and scalable solution.

Future research may explore the generalizability of these findings across more complex structured prediction tasks, larger datasets, and additional adapter architectures. Further investigation into robustness under noisy inputs, multilingual scenarios, and real-world deployment constraints could also provide valuable insights into the practical trade-offs between prompting and parameter-efficient fine-tuning.

7. Patents

This section is not mandatory, but may be added if there are patents resulting from the work reported in this manuscript.

Author Contributions: Conceptualization, R.K. and L.S.; methodology, R.K., L.S.; software, R.K. and L.S.; validation, R.K. and L.S.; formal analysis, R.K.; investigation, R.K. and L.S.; resources, R.K. and L.S.; data curation, R.K. and L.S.; writing—original draft preparation, R.K. and L.S.; writing—review and editing, R.K. and L.S.; visualization, R.K.; supervision, S.B.Š. and I.L.; project administration, S.B.Š. and I.L.; funding acquisition, S.B.Š. and I.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work was (partially) supported by the EC Digital Europe Programme EDIH Adria 2.0 (101256325); SPIN projects IP.1.1.03.0120, IP.1.1.03.0028 and IP.1.1.03.0039; and NextGenerationEU University grants: uniri-iz-25-6, uniri-iz-25-220, IIP_010144, IIP_010136, and UNIN-TEH-25-1-8.

Data Availability Statement: Data are contained within the article.

Acknowledgments: This research was performed using the Advanced computing service provided by University of Zagreb University Computing Centre - SRCE.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

MDPI	Multidisciplinary Digital Publishing Institute
DOAJ	Directory of open access journals
TLA	Three letter acronym
LD	Linear dichroism

Appendix A

Appendix A.1

The appendix is an optional section that can contain details and data supplemental to the main text—for example, explanations of experimental details that would disrupt the flow of the main text but nonetheless remain crucial to understanding and reproducing the research shown; figures of replicates for experiments of which representative data are shown in the main text can be added here if brief, or as Supplementary Data. Mathematical proofs of results not central to the paper can be added as an appendix.

Table A1. This is a table caption.

Title 1	Title 2	Title 3
Entry 1	Data	Data
Entry 2	Data	Data

Appendix B

All appendix sections must be cited in the main text. In the appendices, Figures, Tables, etc. should be labeled, starting with "A"—e.g., Figure A1, Figure A2, etc.

References

1. Ling, C.; Zhao, X.; Lu, J.; Deng, C.; Zheng, C.; Wang, J.; Chowdhury, T.; Li, Y.; Cui, H.; Zhang, X.; et al. Domain specialization as the key to make large language models disruptive: A comprehensive survey. *ACM Computing Surveys* **2025**, *58*, 1–39.
2. Han, Z.; Gao, C.; Liu, J.; Zhang, J.; Zhang, S.Q. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608* **2024**.
3. Karlović, R.; Lorencin, I. Large language models as Retail Cart Assistants: A Prompt-Based Evaluation. *Human Systems Engineering and Design (IHSED2025): Future Trends and Applications* **2025**, *198*.
4. Mundra, N.; Doddapaneni, S.; Dabre, R.; Kunchukuttan, A.; Puduppully, R.; Khapra, M.M. A comprehensive analysis of adapter efficiency. In Proceedings of the Proceedings of the 7th Joint International Conference on Data Science & Management of Data (11th ACM IKDD CODS and 29th COMAD), 2024, pp. 136–154.
5. Hu, Z.; Wang, L.; Lan, Y.; Xu, W.; Lim, E.P.; Bing, L.; Xu, X.; Poria, S.; Lee, R. Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models. In Proceedings of the Proceedings of the 2023 conference on empirical methods in natural language processing, 2023, pp. 5254–5276.
6. Patil, R.; Khot, P.; Gudivada, V. Analyzing LLAMA3 Performance on Classification Task Using LoRA and QLoRA Techniques. *Applied Sciences* **2025**, *15*, 3087.
7. Ding, N.; Qin, Y.; Yang, G.; Wei, F.; Yang, Z.; Su, Y.; Hu, S.; Chen, Y.; Chan, C.M.; Chen, W.; et al. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature machine intelligence* **2023**, *5*, 220–235.
8. Esmaeili, A.; Saberi, I.; Fard, F. Empirical studies of parameter efficient methods for large language models of code and knowledge transfer to r. *Empirical Software Engineering* **2026**, *31*, 30.

9. Razuvayevskaya, O.; Wu, B.; Leite, J.A.; Heppell, F.; Srba, I.; Scarton, C.; Bontcheva, K.; Song, X. Comparison between parameter-efficient techniques and full fine-tuning: A case study on multilingual news article classification. *Plos one* **2024**, *19*, e0301738.
10. Shin, J.; Tang, C.; Mohati, T.; Nayebi, M.; Wang, S.; Hemmati, H. Prompt Engineering or Fine-Tuning: An Empirical Assessment of LLMs for Code. In Proceedings of the 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR). IEEE, 2025, pp. 490–502.
11. Gong, M.; Deng, Y.; Qi, N.; Zou, Y.; Xue, Z.; Zi, Y. Structure-learnable adapter fine-tuning for parameter-efficient large language models. In Proceedings of the International Conference on Artificial Intelligence and Computational Engineering (AICE 2025). IET, 2025, Vol. 2025, pp. 225–230.
12. Fang, X.; Ye, M. Towards Robust Parameter-Efficient Fine-Tuning for Federated Learning. In Proceedings of the The Thirty-ninth Annual Conference on Neural Information Processing Systems, 2025.
13. Chen, S.; Gu, J.; Han, Z.; Ma, Y.; Torr, P.; Tresp, V. Benchmarking robustness of adaptation methods on pre-trained vision-language models. *Advances in Neural Information Processing Systems* **2023**, *36*, 51758–51777.
14. Chen, L.C.; Weng, H.T.; Pardeshi, M.S.; Chen, C.M.; Sheu, R.K.; Pai, K.C. Evaluation of Prompt Engineering on the Performance of a Large Language Model in Document Information Extraction. *Electronics* **2025**, *14*, 2145.
15. Zhu, K.; Wang, J.; Zhou, J.; Wang, Z.; Chen, H.; Wang, Y.; Yang, L.; Ye, W.; Zhang, Y.; Gong, N.; et al. Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts. In Proceedings of the Proceedings of the 1st ACM workshop on large AI systems and models with privacy and safety analysis, 2023, pp. 57–68.
16. Kim, J.; Mao, Y.; Hou, R.; Yu, H.; Liang, D.; Fung, P.; Wang, Q.; Feng, F.; Huang, L.; Khabsa, M. RoAST: Robustifying Language Models via Adversarial Perturbation with Selective Training. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2023, 2023, pp. 3412–3444.
17. Pei, A.; Yang, Z.; Zhu, S.; Cheng, R.; Jia, J. Selfprompt: Autonomously evaluating llm robustness via domain-constrained knowledge guidelines and refined adversarial prompts. In Proceedings of the Proceedings of the 31st International Conference on Computational Linguistics, 2025, pp. 6840–6854.
18. Mu, L.; Chu, G.; Ni, L.; Sang, L.; Wu, Z.; Jin, P.; Zhang, Y. Robustness of Prompting: Enhancing Robustness of Large Language Models Against Prompting Attacks. *arXiv preprint arXiv:2506.03627* **2025**.
19. Sajjadi Mohammadabadi, S.M.; Kara, B.C.; Eyupoglu, C.; Uzay, C.; Tosun, M.S.; Karakuş, O. A survey of large language models: evolution, architectures, adaptation, benchmarking, applications, challenges, and societal implications. *Electronics* **2025**, *14*.
20. Trad, F.; Chehab, A. Prompt engineering or fine-tuning? a case study on phishing detection with large language models. *Machine Learning and Knowledge Extraction* **2024**, *6*, 367–384.
21. Pornprasit, C.; Tantithamthavorn, C. Fine-tuning and prompt engineering for large language models-based code review automation. *Information and Software Technology* **2024**, *175*, 107523.
22. Wang, J.; Albarghouthi, A.; Sala, F. COSMOS: Predictable and Cost-Effective Adaptation of LLMs. *arXiv preprint arXiv:2505.01449* **2025**.
23. Liu, H.; Tam, D.; Muqeeth, M.; Mohta, J.; Huang, T.; Bansal, M.; Raffel, C. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning, 2022, [arXiv:cs.LG/2205.05638].